CSE 3208 — SYSTEM ANALYSIS & DESIGN LAB

RelifMesh

Disaster Relief Coordination System

"One mesh. No duplicates. No household left behind."

Team: Team_Skipper
2101011001 - Abidul Islam, 2101011005 - MD. Kamrul Hassan
2101011013 - Sayeda Mofatteha Ahmed, 2101011038 - Iftekhar Alam Nahid

---

Supervisor: MD Mynoddin, Assistant Professor, Dept. of CSE, RMSTU

Rangamati Science & Technology University

01 Introduction & Problem Statement

Disaster relief coordination for Union Parishads in flood-prone Bangladesh

RelifMesh coordinates relief distribution across Union Parishad officials, NGOs, Upazila officers, and citizens in real time. **August 2024 floods affected 4.5–5.8M people; 92% of mobile towers in Feni went down** — cloud-only tools fail exactly when they're needed most.

1
Duplicate Aid
Same household served multiple times by different agencies

2
Missed Households
Vulnerable families skipped with no audit trail

3
No Real-Time Tracking
Paper-based logs; days of consolidation delay

4
Inter-Agency Opacity
NGOs, government & volunteers operate independently

5
Offline Environments
92% mobile towers failed in Feni 2024 floods

6
No Visibility
Nobody can see which areas still need relief

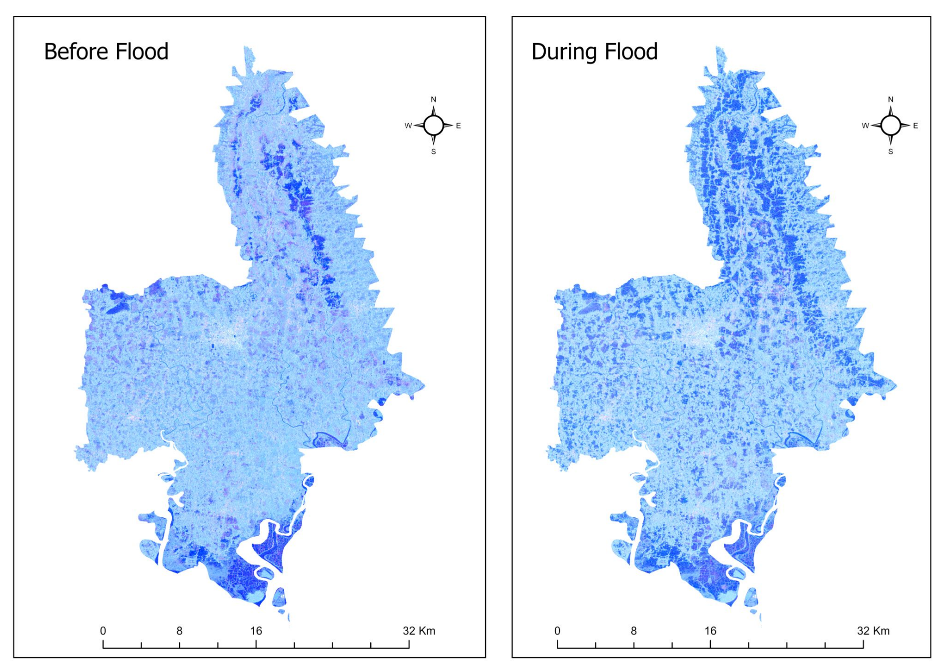
7
Manual Reporting
Slow, error-prone data consolidation

8
Lack of Transparency
Communities cannot verify distribution fairness

02 Project Objectives & Scope

SMART objectives grounded in the Feni Floods 2024–2025 case study

S	**Specific**	Real-time duplicate detection with a 7-day lookback window
M	**Measurable**	95%+ offline sync success; 90%+ duplicate detection accuracy
A	**Achievable**	GPS-based household registration with photo evidence
R	**Relevant**	Transparent public dashboard with an interactive heatmap
T	**Time-bound**	Complete system deployed within a 3-week development cycle



In Scope

- GPS/photo household registration & relief logging
- Rule-based duplicate detection engine
- Offline-first entry with sync & conflict resolution
- Public dashboard, heatmap & pledge coordination

Out of Scope

- Financial transactions & procurement
- AI-based forecasting & national NID integration
- Native mobile apps (PWA only)
- Real-time satellite GIS

03

Stakeholder Analysis

5 key personas across the relief ecosystem, mapped by power and interest

Stakeholder	Role	Power	Interest
Upazila Nirbahi Officer	Strategic oversight	High	High
UP Official	Primary operator	High	High
NGO / Aid Worker	Field partner	Low	High
Citizen / Household	Beneficiary	Low	High
Outside Donor	Supporter	Low	Low
System Administrator	Technical	High	High

Power–Interest Grid

High Power / High Interest

UNO · UP Official · Admin — engage closely

Low Power / High Interest

Citizens · NGO Workers · Donors — keep informed

High Power / Low Interest

Oversight bodies — keep satisfied

Low Power / Low Interest

General public — monitor minimally

04

Functional Requirements

44 functional requirements across 11 feature categories (FR-01 to FR-44)

Category	#	Key Features
Authentication	5	Registration, login, RBAC, profile, password reset
Area Management	3	Geo hierarchy CRUD (Division → Ward), search
Household Mgmt	6	GPS/NID registration, vulnerability flags, CRUD
Distribution	6	Item-level logging, photo/GPS proof, CRUD
Duplicate Alerts	3	7-day detection, resolution workflow, dashboard
Need Assessment	4	Sphere-standard calculation, override, comparison
Pledge Management	5	Multi-source pledges, fulfillment, status lifecycle
Feedback	3	Citizen submission, official response, moderation
Reports	3	PDF generation, CSV export, filtered queries
Sync	3	Offline queue push/pull, conflict resolution
Public Dashboard	3	Aggregate stats, heatmap, transparency view

28 Must Have

8 Should Have

5 Could Have

3 Won't Have

05 Non-Functional Requirements

15 NFRs spanning performance, reliability, security, compliance & usability

Performance & Scalability

- Sub-2s API response time (p95 percentile)
- Support for 500+ concurrent users

Reliability & Availability

- Offline-first PWA for disconnected areas
- 99.5% uptime target with graceful degradation

Security

- RBAC: 6 roles × 12 permissions matrix
- AES-256 encryption; JWT stateless auth

Compliance & Standards

- GDPR-compliant PII handling
- Sphere humanitarian standards for need calc

Usability & Accessibility

- Responsive design for mobile/tablet field use
- Bengali language support; basic digital literacy

06 System Modeling — Data Flow Diagrams

Context diagram (Level 0) and 6 major processes (Level 1)

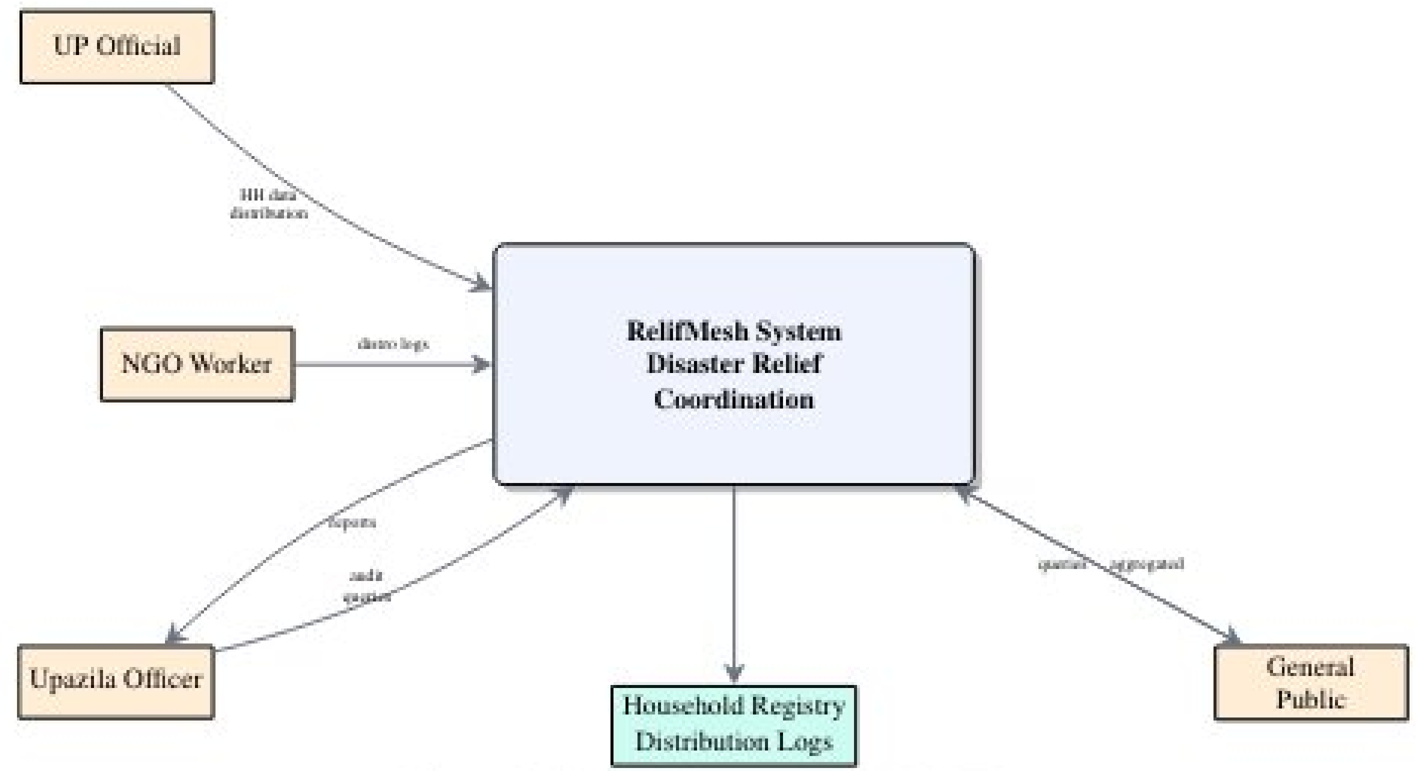


Figure 2: Context Diagram (Level 0 DFD)

Level 1 — 6 major processes: Auth → Household → Distribution → Alert → Need/Report → Sync

07

System Modeling — UML Diagrams

11 diagrams covering all architectural views (draw.io format)

Type	Diagram	Coverage
DFD	Context Diagram (Level 0)	System boundary, 5 external entities
DFD	Level 1 DFD	6 major processes
UML	Use Case Diagram	8 use cases across 4 actor roles
UML	Class Diagram	12 entity classes with relationships
UML	Sequence — Auth & Household Reg.	Registration & login flow
UML	Sequence — Distribution & Alert	Logging + duplicate detection
UML	Sequence — Need Calculation	Sphere-standard computation
UML	Sequence — Pledge Fulfillment	Multi-source pledge lifecycle
UML	Activity — Relief Flow	End-to-end distribution workflow
Other	Entity Relationship Diagram	12 MongoDB collections
Other	Architecture Diagram	3-tier system overview

08 Database Design & ERD

MongoDB — 12 collections, normalized to 3NF

Collection	Key Fields	Purpose
User	name, email, passwordHash, role, jurisdictionId	Auth & RBAC
GeographicArea	name, level (5-tier), parentId, coordinates	Feni geo hierarchy
Household	headName, NID, GPS, familySize, vulnFlags	Beneficiary registry
DistributionLog	householdId, itemId, quantity, GPS, photoUrl	Aid tracking
ReliefPledge	source_type, areaId, itemCategoryId, qty, status	Multi-source pledges
NeedAssessment	areaId, itemCategoryId, calculated_qty, override	Sphere-standard calc
Inventory	itemCategoryId, totalQuantity, distributedQty	Stock management
Feedback	householdId, category, message, response	Citizen feedback
ReliefRequest	citizenId, items[], priority, status	Aid requests
DuplicateAlert	householdId, priorLog, triggeredLog, resolved	Fraud prevention
SyncConflict	offline_id, changes[], merge_strategy	Offline sync
ItemCategory	name, unit, per_person_per_day_qty	Reference data

Key decisions: All schemas normalized to 3NF · 2dsphere indexes on GPS for proximity queries · Document references (not embedded) for scalability

09 System Architecture & Tech Stack

Offline-first 3-tier architecture — PWA + REST API + MongoDB

Client Layer

React 18 PWA · localforage (IndexedDB) · Service Worker

Backend Layer

Express.js · 14 feature modules · Sync Engine

Data Layer

MongoDB Atlas · 12 collections · Geo 2dsphere indexes

Key decisions: Offline-first writes queue in localforage & auto-sync on reconnect · Last-write-wins conflict resolution · Self-seeding DB with Feni data · Role-scoped queries via req.user.jurisdictionId

10 UI/UX Design

Design system, key pages, and interface assets

Typography & Color — DM Serif Display (headings) + Outfit (body); light/dark themes, blue primary

Spacing & Layout — 4px grid scale; sidebar 260px/64px; 64px topbar; fluid content

Key Pages

Landing Page
Public entry — stats, CTA, hero overview

Dashboard
KPI cards, charts, distributions

Public Dashboard
Interactive heatmap with drill-down

Households
Table, search, photo capture

Distributions
Item logging, GPS upload

Need Dashboard
Sphere-calc vs. actual comparison

11 Implementation Plan

30 modules across 6 phases — Agile weekly sprints, 3-week cycle

1 — M1–M6

Data Model & Foundation

Scaffold, auth, geo hierarchy, DB seed, frontend setup

2 — M7–M12

Core Operations

Household CRUD, distribution, alerts, inventory, feedback

3 — M13–M18

Need & Pledge Module

Sphere need engine, pledge lifecycle, reports, heatmap

4 — M19–M23

Public Interface

Landing page, public dashboard, citizen feedback/requests

5 — M24–M27

Offline & Sync

Offline queue, push/pull sync, service worker, PWA manifest

6 — M28–M30

Testing & Docs

Integration tests, security audit, documentation

Methodology: Agile with weekly sprints · Git feature branching · ESLint + Prettier · Peer code reviews

12

Testing & Quality Assurance

47/47 tests passing — Unit, Integration & UAT (Jest + Supertest)

28
Unit Tests

6
Integration Tests

5
UAT Scenarios

47/47
All Passing

Module	Tests	Key Coverage
Auth	5	Register, login, token, invalid credentials
Household	5	CRUD, NID validation, GPS validation
Distribution	5	CRUD, GPS/quantity validation
Feedback	4	Submit, list, respond, moderation
Sync	4	Offline push, pull, conflict resolution
Reports	3	PDF generation, CSV export
Alerts	2	7-day duplicate detection window

13 Security & Access Control

STRIDE threat model · JWT auth · RBAC across 6 roles

Permission	Admin	UNO	UP Official	NGO	Citizen	Anon.
User Mgmt	✓	✓	✓	✓	—	—
Household	—	✓	✓	—	—	—
Distribution	✓	✓	✓	✓	—	—
Reports	✓	✓	—	✓	—	—
Public Dash.	—	—	—	—	—	✓

Middleware Stack

- helmet, cors, express-rate-limit (100/15min)
- express-validator input sanitization
- authenticate.js (JWT) + authorize.js (RBAC)

Data Protection

- GDPR-compliant PII handling; NID encrypted at rest
- AES-256 encryption; JWT expiry configurable (24h)
- STRIDE model: 11 threat rows analyzed

Auth: JWT (jsonwebtoken) Bearer tokens, bcrypt password hashing, 6 roles × 12 permissions matrix, centralized error handling with no information leakage.

14

Deployment & Project Management

Zero-cost cloud deployment · Team_Skipper · 4-week timeline

Deployment

Frontend	Netlify — auto-deploy from git
Backend	Railway — Node.js Express API
Database	MongoDB Atlas — free M0 cluster

Maintenance: daily health checks, weekly sync review, monthly patches, quarterly audits.

Team_Skipper

Abidul Islam	Backend Lead
MD. Kamrul Hassan	Frontend Lead
Sayeda M. Ahmed	Documentation
Iftekhar A. Nahid	Diagrams & DevOps

Timeline

W1	Requirements, architecture, DB design
W2	Core implementation, API development
W3	Frontend, integration, testing
W4	Documentation, submission, defense prep

15

Project Statistics & Conclusion

14/14 SAD Modules	**44** Functional Reqs	**47/47** Tests Passing	**11** System Diagrams
12 DB Collections	**50+** API Endpoints	**6** Impl. Phases	**8.5/10** Assessment Score

Conclusion

- RelifMesh addresses duplicate aid, invisible need, and offline unusability with a single coordination layer.
- Offline-first PWA architecture keeps operations running through network outages.
- Built incrementally over 6 phases / 30 modules using Agile methodology; validated by 47/47 passing tests.
- All project objectives met: registration, duplicate detection, offline sync, dashboard, RBAC, need assessment, heatmap.

"One mesh. No duplicates. No household left behind."